

Vibe Coding & AI-Assisted Development Syllabus

Archer Infotech, Kothrud Pune | Classroom & live-online batches | Updated September 2026

Course Overview

The **Vibe Coding & AI-Assisted Software Development** course teaches students how to build modern software applications using natural-language instructions, AI coding assistants, and coding agents.

Students learn how to convert an idea into requirements, generate application code, understand and improve AI-generated code, debug problems, create tests, work with databases and APIs, use Git and GitHub, and finally deploy a complete application.

The course focuses on **responsible AI-assisted development**, where AI helps accelerate software development while the developer remains responsible for understanding, validating, testing, and securing the generated code.

Course Objectives

By the end of this course, students will be able to:

- Understand Vibe Coding and AI-assisted software development
- Convert application ideas into clear software requirements
- Write effective prompts for coding agents
- Work with tools such as Cursor, Claude Code, OpenAI Codex, and GitHub Copilot
- Generate frontend and backend applications using AI
- Design databases with AI assistance
- Create and consume REST APIs
- Debug AI-generated code
- Generate and execute software tests
- Review and refactor AI-generated code
- Use Git and GitHub for source control
- Apply secure coding practices
- Deploy AI-generated applications
- Build a complete application using an AI-assisted development workflow

Module 1: Introduction to Vibe Coding

1.1 What is Vibe Coding?

- Introduction to Vibe Coding
- Evolution of software development
- Traditional Coding vs AI-Assisted Coding
- Vibe Coding vs Traditional Programming
- Vibe Coding vs Low-Code / No-Code
- Vibe Coding vs Agentic Coding
- Role of developers in AI-assisted development

1.2 How AI Coding Assistants Work

- Introduction to Large Language Models
- How coding models generate code
- Understanding prompts and context
- AI coding assistants
- AI coding agents
- Model context
- Repository context

- Agent instructions
- Tool calling

1.3 Advantages of Vibe Coding

- Faster prototyping
- Reduced development time
- Faster debugging
- Automated documentation
- Code generation
- Code explanation
- Test generation
- Rapid experimentation

1.4 Limitations of Vibe Coding

- AI hallucinations
- Incorrect code generation
- Security vulnerabilities
- Dependency hallucination
- Poor architecture
- Lack of code understanding
- Over-dependence on AI
- Importance of human review

Module 2: AI Coding Tools

2.1 Introduction to AI Coding Tools

Overview of modern AI coding environments.

Tools covered may include:

- Cursor
- Claude Code
- OpenAI Codex
- GitHub Copilot
- ChatGPT
- Visual Studio Code
- Terminal-based coding agents

2.2 Cursor

- Installing Cursor
- Understanding the Cursor interface
- AI Chat
- Inline Editing
- Code Completion
- Agent Mode
- Repository context
- Multi-file code generation
- Debugging using Cursor
- Project rules
- Working with existing applications

2.3 Claude Code

- Introduction to Claude Code
- Terminal-based AI coding
- Understanding repository context

- Generating new features
- Editing multiple files
- Debugging applications
- Running commands
- Refactoring existing projects
- Working with project instructions

2.4 OpenAI Codex

- Introduction to Codex
- AI-assisted development
- Code generation
- Repository understanding
- Bug fixing
- Code review
- Code refactoring
- Task-based software development

2.5 GitHub Copilot

- Installing GitHub Copilot
- Code suggestions
- Copilot Chat
- Code explanation
- Generating functions
- Writing tests
- Debugging
- Code review assistance

Module 3: Prompt Engineering for Coding

3.1 Writing Better Coding Prompts

- Understanding developer intent
- Providing sufficient context
- Giving constraints
- Specifying technologies
- Providing acceptance criteria
- Asking AI to explain generated code

3.2 Requirement-Based Prompting

Learn how to convert a simple idea such as:

> Build a patient management application.

into structured requirements.

Topics include:

- Functional requirements
- Non-functional requirements
- User roles
- Application workflows
- Data requirements
- Validation requirements
- UI requirements

3.3 Structured Prompting

Writing prompts containing:

- Role
- Objective
- Context
- Technology stack
- Functional requirements
- Constraints
- Expected output
- Acceptance criteria

3.4 Iterative Prompting

- Initial generation
- Reviewing output
- Providing feedback
- Correcting mistakes
- Adding features
- Refining UI
- Refactoring code

Module 4: Software Development with AI

4.1 Software Development Lifecycle

Understanding:

Idea

- Requirement
- Design
- Development
- Testing
- Review
- Deployment

4.2 AI-Assisted Development Workflow

Recommended workflow:

Idea

- Requirements
- Architecture
- Task Breakdown
- Prompt
- Generate
- Review
- Test
- Debug
- Refactor
- Deploy

4.3 Application Planning

Using AI for:

- Requirement gathering
- Feature identification
- User stories
- Acceptance criteria
- Application architecture
- Technology selection
- Database design
- API planning
- Task breakdown

Module 5: Frontend Development with AI

5.1 Web Development Fundamentals

Basic understanding of:

- HTML
- CSS
- JavaScript

5.2 Frontend Framework Introduction

Introduction to:

- React
- Components
- Props
- State
- Events
- Forms

5.3 Building UI with AI

Generate:

- Navigation bars
- Login pages
- Registration pages
- Forms
- Tables
- Dashboards
- Search interfaces
- CRUD screens
- Responsive layouts

5.4 Form Development

- Input controls
- Form validation
- Required fields
- Error messages
- Dropdowns
- Date fields
- Form submission

5.5 AI-Assisted UI Improvement

Using AI to improve:

- Layout

- Responsiveness
- Accessibility
- UX
- Error handling
- User feedback
- Component reuse

Module 6: Backend Development with AI

6.1 Backend Fundamentals

Understanding:

- Client
- Server
- Request
- Response
- HTTP
- REST APIs

6.2 Building APIs

Using AI to generate:

- GET APIs
- POST APIs
- PUT APIs
- DELETE APIs

6.3 CRUD Operations

Implement:

- Create
- Read
- Update
- Delete

6.4 API Validation

- Input validation
- Error handling
- HTTP status codes
- Validation messages
- Exception handling

6.5 API Documentation

Introduction to:

- Swagger
- OpenAPI
- API documentation

Module 7: Database Development with AI

7.1 Database Fundamentals

- What is a database?
- Relational databases
- NoSQL databases
- Tables

- Rows
- Columns
- Primary keys
- Foreign keys

7.2 Database Design

Using AI to generate:

- Entities
- Tables
- Relationships
- Database schema
- ER diagrams

7.3 SQL Fundamentals

Basic SQL:

- CREATE
- INSERT
- SELECT
- UPDATE
- DELETE

7.4 Database Integration

Connecting the application with:

- MySQL
- PostgreSQL
- MongoDB

7.5 AI-Assisted Query Generation

Using AI for:

- SQL generation
- Query explanation
- Query optimization
- Debugging database problems

Module 8: Authentication and Authorization

8.1 Authentication

Understanding:

- Login
- Logout
- Registration
- Password storage
- Sessions
- Tokens

8.2 Authorization

Understanding:

- User roles
- Permissions
- Role-based access control

Example roles:

- Admin
- Patient
- Chemist

8.3 AI-Assisted Authentication Development

Using coding agents to generate:

- Login screens
- Authentication APIs
- Authorization middleware
- Protected routes

Module 9: Debugging with AI

9.1 Understanding Software Errors

Different types of errors:

- Syntax errors
- Runtime errors
- Logical errors
- API errors
- Database errors
- UI errors

9.2 AI-Assisted Debugging

Providing AI with:

- Error messages
- Stack traces
- Logs
- Code snippets
- Expected behaviour

9.3 Debugging Workflow

Error

- Understand
- Reproduce
- Analyze
- Fix
- Test
- Verify

9.4 Common AI-Generated Code Problems

- Wrong imports
- Missing packages
- Incorrect API usage
- Wrong variable names
- Broken database queries
- Incorrect assumptions
- Unsupported libraries

Module 10: Testing with AI

10.1 Software Testing Fundamentals

- Why testing is required
- Functional testing
- Unit testing
- Integration testing
- UI testing

10.2 AI-Generated Tests

Use AI to generate:

- Unit test cases
- API test cases
- Validation test cases
- Edge cases
- Negative test cases

10.3 Test Automation Introduction

Introduction to tools such as:

- PyTest
- Jest
- Playwright

10.4 AI-Assisted Test Debugging

- Understanding failed tests
- Identifying root causes
- Updating tests
- Fixing application code

Module 11: Code Review and Refactoring

11.1 Understanding AI-Generated Code

Students must learn to ask:

- What does this code do?
- Why was this approach used?
- Is this code secure?
- Can this code be simplified?
- Is there duplicated code?
- Are errors handled properly?

11.2 AI-Assisted Code Review

Use AI to review:

- Code quality
- Performance
- Security
- Maintainability
- Readability

11.3 Refactoring

AI-assisted techniques for:

- Removing duplicate code
- Improving naming
- Breaking large functions
- Creating reusable components

- Improving API structure
- Optimizing database access

Module 12: Git and GitHub with AI

12.1 Git Fundamentals

- Repository
- Commit
- Branch
- Merge
- Clone
- Pull
- Push

12.2 GitHub

- Creating repositories
- Pushing code
- Branching
- Pull requests
- Code review

12.3 AI-Assisted Git Workflow

Using AI for:

- Commit messages
- Pull-request descriptions
- Change summaries
- Code review
- Merge-conflict explanation

Module 13: Secure Vibe Coding

13.1 Secure Coding Fundamentals

Understand common risks:

- Hard-coded passwords
- Exposed API keys
- SQL Injection
- Cross-Site Scripting
- Authentication vulnerabilities
- Authorization vulnerabilities

13.2 Secrets Management

Correct use of:

- .env
- Environment variables
- .gitignore
- Secret-management practices

13.3 AI Coding Agent Permissions

Understand risks related to:

- File access
- Terminal access
- Command execution

- Database access
- Production access

13.4 Responsible Vibe Coding

Follow the workflow:

Generate → Understand → Review → Test → Secure → Approve

Never blindly accept AI-generated code.

Module 14: Deployment with AI

14.1 Introduction to Deployment

Understand:

Development

→ Testing

→ Staging

→ Production

14.2 Preparing Applications for Deployment

- Environment configuration
- Production builds
- Environment variables
- Database configuration
- Logging
- Error handling

14.3 Deployment Platforms

Introduction to platforms such as:

- Vercel
- Netlify
- Render
- Railway
- Cloud platforms

14.4 AI-Assisted Deployment

Using AI for:

- Deployment instructions
- Configuration files
- Docker files
- Environment configuration
- Deployment troubleshooting

Module 15: Advanced Vibe Coding

15.1 Context Engineering

Understand how coding agents use:

- Prompts
- Files
- Repository structure
- Documentation
- Project instructions

- Previous conversations

15.2 Project Rules

Creating project-specific instructions covering:

- Coding standards
- Architecture
- Folder structure
- Testing requirements
- Security requirements

15.3 Specification-Driven Development

Instead of directly saying:

> Build this application.

Students learn to create:

- Product requirements
- Functional requirements
- Architecture
- Database design
- API specification
- UI requirements
- Acceptance criteria
- Implementation tasks

Then ask the coding agent to implement them.

15.4 Agentic Coding

Introduction to:

- Planning agents
- Coding agents
- Testing agents
- Review agents
- Documentation agents

15.5 MCP Fundamentals

Introduction to Model Context Protocol concepts:

- Connecting tools
- External context
- Development tools
- Database tools
- Documentation systems

Archer Responsible Vibe Coding Workflow

Throughout the course, students follow the following process:

IDEA

Define what needs to be built.

↓

REQUIREMENT

Write clear functional requirements.

↓

SPECIFICATION

Define workflows, architecture and acceptance criteria.

↓

PLAN

Ask AI to break the implementation into tasks.

↓

GENERATE

Generate application code.

↓

UNDERSTAND

Understand the generated implementation.

↓

TEST

Test functionality and edge cases.

↓

REVIEW

Review code quality and architecture.

↓

SECURE

Check security and secrets.

↓

REFACTOR

Improve code quality.

↓

DEPLOY

Deploy the finished application.

Final Mini Project**Patient & Chemist Management System**

Students will build a complete **Patient & Chemist Management System** using Vibe Coding and AI-assisted software-development techniques.

The purpose of the project is to apply all concepts taught during the course.

Project Scenario

A medical application is required where patients can maintain basic medicine requirements and communicate prescription-related information to chemists.

The application will support multiple user roles and allow users to manage patients, medicines, prescriptions, chemists and medicine requests.

User Roles

Patient

A patient should be able to:

- Register
- Login
- Update profile
- View prescriptions
- Search medicines
- View medicine information
- Submit medicine requests
- View order/request status
- View previous medicine requests

Chemist

A chemist should be able to:

- Login
- Manage chemist profile
- View patient medicine requests
- Search medicines
- Update medicine availability
- Confirm medicine requests
- Update request status

Admin

Admin should be able to:

- Manage patients
- Manage chemists
- Manage medicines
- View medicine requests
- Activate/deactivate users
- View dashboard information

Suggested Database Entities

Students can design the database using entities such as:

Patient

- patient_id
- name
- email
- phone
- date_of_birth
- address

Chemist

- chemist_id
- shop_name
- owner_name
- email
- phone
- address

Medicine

- medicine_id

- medicine_name
- manufacturer
- description
- price
- stock_quantity

Prescription

- prescription_id
- patient_id
- doctor_name
- prescription_date
- notes

PrescriptionMedicine

- prescription_id
- medicine_id
- dosage
- duration

MedicineRequest

- request_id
- patient_id
- chemist_id
- request_date
- status

MedicineRequestItem

- request_item_id
- request_id
- medicine_id
- quantity

Application Screens

Students can build screens such as:

Common Screens

- Home
- Login
- Registration
- Forgot Password

Patient Screens

- Patient Dashboard
- Patient Profile
- Medicine Search
- Medicine Details
- Prescription List
- Medicine Request
- Request History

Chemist Screens

- Chemist Dashboard
- Chemist Profile
- Medicine Inventory

- Patient Requests
- Request Details
- Request Status Update

Admin Screens

- Admin Dashboard
- Patient Management
- Chemist Management
- Medicine Management
- Request Management

APIs to Build

Example APIs:

Authentication

- Register Patient
- Register Chemist
- Login
- Logout

Patient APIs

- Create Patient
- Get Patient
- Update Patient
- Get Patient Requests

Chemist APIs

- Create Chemist
- Get Chemist
- Update Chemist
- View Patient Requests

Medicine APIs

- Add Medicine
- Get Medicines
- Search Medicine
- Update Medicine
- Delete Medicine

Prescription APIs

- Create Prescription
- Get Patient Prescriptions
- Add Medicine to Prescription

Medicine Request APIs

- Create Medicine Request
- Get Medicine Request
- Update Request Status
- View Request History

AI-Assisted Project Development Process

Students must not directly ask AI to generate the entire project.

They should complete the project through structured stages.

Stage 1 – Idea

Describe the Patient & Chemist application.

Stage 2 – Requirements

Ask AI to generate:

- Functional requirements
- User roles
- Application workflows
- Acceptance criteria

Stage 3 – Architecture

Ask AI to propose:

- Frontend architecture
- Backend architecture
- Database architecture

Stage 4 – Database Design

Generate:

- Entities
- Relationships
- ER diagram
- Database schema

Stage 5 – API Design

Generate:

- REST API list
- Request models
- Response models
- Validation requirements

Stage 6 – UI Design

Generate:

- Application layout
- Navigation
- Forms
- Dashboards
- Tables

Stage 7 – Application Development

Use coding agents to implement:

Frontend

+

Backend

+

Database

Stage 8 – Testing

Generate and execute:

- Unit tests

- API tests
- Validation tests
- UI tests

Stage 9 – Debugging

Use coding agents to identify and resolve application errors.

Stage 10 – Code Review

Use AI to inspect:

- Code quality
- Architecture
- Security
- Performance
- Maintainability

Stage 11 – Refactoring

Improve the implementation based on the review.

Stage 12 – Deployment

Deploy the application and verify the production build.

Final Project Deliverables

Each student should submit:

- Software requirement document
- Application architecture
- Database schema
- ER diagram
- API documentation
- Source code
- GitHub repository
- Test cases
- AI code-review report
- Working application
- Deployment URL
- Project presentation

Learning Outcome

After completing this project, students should be capable of taking an application idea and independently following:

Idea → Requirements → Architecture → Prompt → Generate → Understand → Test → Debug → Review → Secure → Deploy

using modern AI coding tools.

Recommended Course Duration

6–8 Weeks

Suggested training time:

45–60 Hours

Recommended distribution:

- Concepts and demonstrations – 30%

- Guided coding exercises – 30%
- Hands-on assignments – 20%
- Patient & Chemist Project – 20%

Target Audience

This course is suitable for:

- Engineering students
- BCA / MCA students
- Diploma students
- Fresh graduates
- Software developers
- Full-stack developers
- Software testers
- Technical professionals
- Entrepreneurs
- Anyone interested in AI-assisted application development

Prerequisites

Basic computer knowledge is required.

Basic programming knowledge is helpful but not mandatory.

Students with programming knowledge will be able to explore more advanced use cases involving architecture, debugging, testing and code review.

Tools & Technologies

Students may work with:

- Cursor
- Claude Code
- OpenAI Codex
- GitHub Copilot
- ChatGPT
- Visual Studio Code
- Git
- GitHub
- HTML
- CSS
- JavaScript / TypeScript
- React
- REST APIs
- SQL / MongoDB
- Testing frameworks
- Deployment platforms

Course Completion Outcome

On successful completion of the course, students will understand how to use modern AI coding assistants and coding agents as part of a professional software-development workflow.

Rather than simply generating code, students will learn how to **plan, build, understand, test, review, secure and deploy AI-assisted software applications.**

Enquire or book a free demo session. Archer Infotech, Flat No. 12, Divyadarshan Housing Society, behind Kothrud Bus Stand Road, Kothrud, Pune 411038. Phone +91 9850 678451. Email info@archerinfotech.in. Web archerinfotech.in.

ARCHER INFOTECH PUNE
archerinfotech.in